RED_TEAM_REVIEW

Helios â€” Red-Team Review (Adversarial Stress Test)

`RED_TEAM_REVIEW.md` Â· Version v1.0 Â· 2026-06-03 Â· *Mandate: destroy Helios. No redesign. Stress-test the existing architecture.*

Method. Four principal critics â€” Architect, Analytics Engineer, Product Analyst, AI Engineer â€” independently attacked the live artifacts (Bible, DATA_MODEL, DBT_GUIDE, `semantic_layer.yaml`, MCP/AGENT architecture, `scenarios.yaml`, DEVELOPMENT_PLAN). Findings are brutal but **grounded and cited**. Each is tagged **Fundamental** (cannot be fixed without abandoning the premise) or **Fixable** (sloppiness). This front matter synthesizes; Â§1â€“Â§4 are the full reviews.

Severity tally: 12 Critical Â· 19 High Â· 14 Medium (45 total). **Fundamental â‰ˆ 25 / Fixable â‰ˆ 20.** The fixable ones are the *least* of Helios's problems.

The Kill-List (ranked, deduplicated across critics)

1. **CRITICAL â€” Nothing is built; every quantitative claim is an unearned target.** "â‰¥85% accuracy," "<5 min/run," "0 hallucinated columns," "production-grade," the Â§20.9 results table (0.882, 0.441â€¦) â€” all are *design intent printed as results*. The only thing "validated" is a YAML referential-integrity **lint**. *(Architect + AI Eng headlines; all 4.)*
2. **CRITICAL â€” The headline benchmark is circular.** Bible Â§20.2 admits the labels are "computed analytically by the same decomposition algebra Helios is graded on." On injected aggregates the mix/rate/interaction split is exact **by construction** â†’ the 85% measures whether the system can run its own equation twice. It is a unit test of arithmetic, not diagnostic accuracy. *(AI Eng C; Product Analyst H; Analytics Eng C.)*
3. **CRITICAL â€” "Diagnose WHY" is an overclaim; the engine computes WHERE.** Mix-vs-rate is *arithmetic attribution* (which segment, mix or rate), not *causal inference* (deploy, price, broken SDK, competitor, holiday â€” all outside the dimensional space). Proof in its own data: scenario **S021** injects only a `0.62` rate multiplier yet labels it an "iOS WebView

payment-SDK regression" â€" a cause found **nowhere in the dataset**. The causal layer is asserted, never derived. *(Product Analyst C; AI Eng C; Analytics Eng C.)*

4. **CRITICAL â€" The static, frozen, 3-month dataset is incompatible with the entire autonomous premise.** "Always-on," "scheduled cron," "anomaly detection over time," "source freshness," "day_30 retention," "forecasting," "revenue-at-risk" â€" every run sees the *same frozen export*. The product's core value (continuous autonomous diagnosis) is **undemonstrable on the data it is built on**. *(Product Analyst C; Architect H; Analytics Eng H.)*
5. **CRITICAL â€" "Trust" rests on verification that isn't verification.** The Critic and the faithfulness check are *stochastic LLMs with no oracle* grading other stochastic LLMs; "adversarial verification" is sampling, not proof. Meanwhile "0 hallucinated columns" guards the *cheap* failure (a bad column name) and does nothing about the *expensive* one (a confident wrong causal story over correct numbers). The "correct and trusted" thesis is the weakest part. *(AI Eng C; Architect MÃ—2.)*
6. **CRITICAL â€" The agentic apparatus is grossly disproportionate to the problem.** 5 MCP servers + 7 agents (3 Opus) + an FSM + a hypothesis tree + memory + a vector storeâ€¦ to deliver a `GROUP BY` + a subtraction that `decompose_change` does in ~14 lines over a daily fact with **three usable dimensions**. *(Architect C; AI Eng M; Analytics Eng M; Product Analyst H.)*
7. **CRITICAL â€" The decomposition mart cannot represent the segments the eval grades against** (and the documented `fct_funnel` lacks the dimensions the semantic layer/hypothesis-tree drill). The "wide-fact" assumption contradicts the documented columns. *(Analytics Eng C; Architect C.)*
8. **HIGH â€" "Deterministic FSM with model-driven nodes" is a contradiction** that launders stochastic choices (which slice to drill, which story to tell) as determinism. Determinism covers only the SQL/stats, never the answer. *(Architect C; AI Eng H.)*
9. **HIGH â€" `revenue-at-risk` is a false-persistence counterfactual sold as a hard dollar figure** â€" it assumes the degraded rate would otherwise have held at its t0/forecast value, which is routinely false. *(Product Analyst C.)*
10. **HIGH â€" Injected anomalies don't resemble real ones; seasonality/data-quality buckets leak answers through pre-seeded memory.** The benchmark only contains anomalies the algebra can already see, and the Critic "refutes" seasonality using a calendar that was seeded with the answer. *(AI Eng C+H.)*
11. **HIGH â€" The insight-to-action gap is misdiagnosed; the ROI story is unfounded.** The bottleneck is usually deciding/prioritizing/shipping â€" which Helios does not do â€" not the speed of RCA; "days to minutes" assumes RCA *volume* is the cost driver. *(Product Analyst HÃ—2.)*
12. **HIGH â€" Build-vs-buy and cost/latency are unanswered.** GA4/Amplitude/Mixpanel/Looker already ship anomaly detection + segmentation; the

"governed + adversarial-eval" moat is thin and copyable. And 7 agents + tree + Critic re-query loop credibly fitting <5 min / â‰¤5 GiB is **asserted, never measured**. *(Architect HÃ—2; Product Analyst M; AI Eng H.)*

Cross-cutting themes (where multiple critics converged â€” the real fatal patterns)

- **Self-referential proof.** Items 1, 2, 5, 10 are one disease: Helios grades and "verifies" itself with its own machinery (its algebra defines the labels; LLMs check LLMs; seeded memory answers the seasonality bucket; the results table is fabricated). There is **no independent oracle anywhere in the trust story.**
- **WHERE masquerading as WHY.** Items 3, 9 + the AE's "arithmetic attribution" finding: the product's defining verb ("diagnose why") describes something the engine cannot do; it relabels arithmetic movement as causation, and even its benchmark labels smuggle in causes the data lacks.
- **A live-product narrative on dead data.** Item 4 + freshness/retention/forecasting/experiment findings: autonomy, scheduling, monitoring, retention, and "designs experiments" are all theater on a frozen 3-month obfuscated export with no live funnel and no one to action a brief.
- **Complexity committed, value hypothetical.** Items 1, 6, 8 + MCP/memory/maintenance findings: maximal architecture (and a multi-tenant/warehouse-agnostic roadmap) sits atop *zero running code*, with a one-person maintenance surface that the "single source of truth" already can't keep consistent (registry filename, schema keys, scenario count, wide-fact dims all drift across docs).

Fundamental vs Fixable

Fundamental (gut the thesis â€” cannot be fixed without changing what Helios *is*): the circular benchmark; WHERE-not-WHY; autonomy-on-frozen-data; verification-by-LLM; apparatus-vs-problem disproportion; deterministic-FSM contradiction; revenue-at-risk counterfactual; build-vs-buy moat; the data simply cannot support day_30 retention / live anomalies / causal claims.

Fixable (embarrassing, not fatal): the registry filename + schema-key + scenario-count + wide-fact drifts; the broken cohort SQL + duplicated retention numerators; `MAX` / `ANY_VALUE` corrupting slice keys; the 47-metric bloat (CAC-proxy with no cost data); the strawman 45% baseline; MCP/vector-store over-abstraction.

Verdict

As an **engineering portfolio artifact**, the *depth* is real. As the **product it claims to be**, the three most-marketed claims â€" *autonomous diagnosis of WHY*, *proven to 85%*, *correct and trusted* â€" are each **fundamentally unsupported**: the first by the dataset, the second by circular self-grading, the third by stochastic self-verification, and **all three by the fact that none of it has ever run.** The complexity is fully paid for; the value is entirely promissory.

Principal Architect Review

Headline kill. Helios is a 5-MCP-server / 7-agent / FSM / vector-store / memory-store apparatus erected to answer a question â€" "did this rate move because behavior changed or because traffic mix changed?" â€" that the system's own `stats-mcp.decompose_change` answers in **20 lines of arithmetic** (MCP_ARCHITECTURE.md Â§9, lines 274â€"288) over a single daily fact table (`fct_daily_funnel`) that has **exactly three usable dimensions** (`device_category` , `country` , `channel_key` ; Bible Â§8.3). Three Opus agents, a best-first hypothesis tree, an adversarial Critic re-query loop, BigQuery-plus-vector memory, and a deterministic FSM are wrapped around a `GROUP BY` and a subtraction. The entire agentic edifice is a delivery mechanism for a query a competent analyst writes â€" and a notebook runs â€" in minutes. Nothing here is built; the complexity is all committed, none of the value is.

[CRITICAL] The agentic apparatus is grossly disproportionate to the problem it solves

Claim attacked. "seven agents in a plan-execute-critique loop" with three on Opus (AGENT_ARCHITECTURE.md Â§3), five MCP servers (MCP_ARCHITECTURE.md Â§1), a memory store + vector store (Bible Â§22; DEPENDENCY_MAP T7), and a deterministic FSM (AGENT_ARCHITECTURE.md Â§1) â€" all to "diagnose WHY an e-commerce funnel moved." **Why it fails.** The thesis centerpiece is the mix/rate/interaction identity, and the reference implementation of it is `decompose_change` : 14 lines of Python (MCP_ARCHITECTURE.md Â§9). The grain it runs over, `fct_daily_funnel` , is one row per `(date, channel, device_category, country, is_new_user)` (Bible Â§8.3) â€" a table small enough to pull into pandas. The honest scope of the diagnosis is: for each canonical metric, run the decomposition across â‰¤3 dimensions, rank by rate-effect, attach a z-test and a dollar figure. That is a single notebook cell and one `stats` call. Wrapping it in Orchestratorâ†'Monitorâ†'Decomposeâ†'Diagnoseâ†'Criticâ†'Prescribeâ†'Narrator, each a separate LLM with its own system prompt, allow-list, retry policy, and typed envelope (AGENT_ARCHITECTURE.md Â§6), is architecture cosplay. The agent count is justified nowhere by

problem difficulty â€” only by the desire to *look* like a multi-agent system. **Fundamental.** The problem is too small for the machine.

[CRITICAL] "Deterministic finite state machine" is a label laundering stochastic LLMs as deterministic

Claim attacked. "Helios is **not** an autonomous free-roaming agent. It is a **deterministic finite state machine** â€¦ transitions are auditable and reproducible" (AGENT_ARCHITECTURE.md Â§1). The reliability section then concedes "LLM nodes are run at `temperature 0` but are **not** byte-deterministic" (Â§11). **Why it fails.** The FSM determinism covers only the edges â€” PLANâ†'MONITORâ†'DECOMPOSE â€” which are trivial and never the source of error. Every decision that *matters* lives inside a node and is made by a sampling LLM: which slice Diagnose drills (Â§6.4 "drill the slice with the largest rate-effect first" â€ chosen by the model), which of four refutations the Critic pursues and whether it returns PASS/DOWNGRADE/DROP (Â§6.5), which story the Narrator tells (Â§6.7). Calling the wrapper "deterministic" while the wrapped reasoning is non-reproducible (NFR-D2 demands "the same findings set" from a pinned run â€ unverifiable and likely false for an Opus tree search) is the central architectural sleight of hand. "Reproducible transitions" over irreproducible content is reproducibility theater. **Fundamental.** Determinism is asserted of the shell, not the brain.

[CRITICAL] The wide-fact assumption is already broken: the dimensions the agents are told to drill do not exist on the fact

Claim attacked. `list_dimensions()` "MUST return exactly the canonical dimensions" â€ 16 of them including `operating_system`, `browser`, `region`, `source`, `medium`, `campaign`, `landing_page`, `item_category`, `item_name` (FR-A7, Bible Â§6.A); Diagnose expands "along the next canonical dimension" to `MAX_DEPTH=3` (AGENT_ARCHITECTURE.md Â§6.4, Â§9). **Why it fails.** `fct_daily_funnel` carries only `channel_key`, `device_category`, `country`, `is_new_user` (Bible Â§8.3, verified on the artifact). `fct_funnel` adds nothing beyond `device_category` / `country` / `channel_key`. So 11 of the 16 promised dimensions have **no column on the fact the Monitor/Decompose/eval injector all read** (DEPENDENCY_MAP A4.7: "Primary feed for Monitor + Decompose + eval injector"). A best-first tree told to drill 3 dimensions deep will exhaust the real dimensionality at depth â‰ˆ2 and then either fail on `UnknownDimension` or silently never explore `landing_page` / `source` â€ the exact slices where real funnel root causes live. The architecture's headline capability (deep dimensional RCA) is contradicted by its own data model. **Fundamental.** The fact is narrow; the agent contract assumes it is wide.

[HIGH] The guardrail chain is a deep coupled pipeline whose every link is a single point of failure

Claim attacked. "Every governed datapoint flows through this fixed sequence" â€” `build_query â†’ dry_run â†’ run_query â†’ reconcile â†’ stats â†’ Critic â†’ render_brief` enforced by tool preconditions I1â€“I4 (MCP_ARCHITECTURE.md Â§2). **Why it fails.** This is a 7-stage synchronous chain where each stage can hard-fail the datapoint: `NotDryRunFirst` if hashing normalization drifts (the `sha256` gate normalizes whitespace+case, Â§9 line 246 â€” any agent that pretty-prints SQL differently between dry_run and run_query silently breaks I1); `ByteBudgetExceeded` forces the agent to "narrow scope" mid-tree (a control-flow side effect that mutates the analysis); `reconcile â‰¤0.5%` (G4) can fail a perfectly correct finding on floating-point/rounding drift; `UnknownDimension` is "a hard stop for that branch" (Â§11). Each guardrail is individually reasonable; chained synchronously inside an LLM tool loop with â‰¤3 retries and a 5-minute wall-clock, they compound into a brittle pipeline whose failure modes (G3 hash miss, G4 drift, budget mid-run rescope) are all *silent quality degraders*, not loud errors. The operational burden of debugging "why did finding F-2021-0042 get dropped" across 7 servers and 7 agents is enormous for the one person who must run this. **Fixable.** Collapse stages; but coupling is designed-in.

[HIGH] The Criticâ†’Diagnose re-query loop multiplied by the hypothesis tree is an unbounded-in-practice cost/latency bomb

Claim attacked. "<5 min/run" and "â‰¤5 GiB/run" (Bible Â§5.2, NFR-P1/C1) with `MAX_BRANCHING=4`, `MAX_DEPTH=3`, `MAX_REFUTE_ROUNDS=2` (AGENT_ARCHITECTURE.md Â§9), three Opus agents, and a Critic that "can re-query to refute" (MCP_ARCHITECTURE.md Â§10). **Why it fails.** Best-first over 4 branches Ã— 3 depth is up to 4Â³ = 64 nodes, each doing `build_queryâ†’dry_runâ†’run_queryâ†’significance_testâ†’decompose_changeâ†’reconcile` (Â§6.4) â€” that is hundreds of MCP round-trips and Opus turns. Every candidate that survives goes to the Critic (Opus), which itself runs `run_query` confound probes and `significance_test` on holdouts (Â§6.5), and on DOWNGRADE bounces **back to Diagnose for a targeted re-query** up to 2Ã— (Â§5.1 transition table). The compounding is `nodes Ã— candidates Ã— (1 + refute_rounds)` Opus invocations plus their tool fan-out. Asserting this finishes in <5 minutes and under 5 GiB with three Opus agents and a tree search is a target with no measurement behind it (DEPENDENCY_MAP: "The only artifact that currently exists is the Bible â€¦ everything else is remaining"). The sample results table (Bible Â§20.9, "4m12s/run, 2.1 GB") is a fabricated illustration, not a measurement â€” there is no runner to produce it. **Fixable.** Caps exist, but the budget is unvalidated and optimistic.

[HIGH] Build-vs-buy: the off-the-shelf stack does 80% of this with none of the maintenance surface

Claim attacked. "Anti-product stance â€¦ not a BI dashboard, not an ad-hoc SQL chatbot" (Bible Â§4.4) â€” positioned as if no tool covers governed-metric funnel diagnosis. **Why it fails.** A

governed semantic layer (the actual keystone, A5.1) + scheduled anomaly detection + funnel decomposition is largely a solved commodity: Lightdash/Cube/dbt-MetricFlow give you the governed metric layer and SQL generation; Amplitude/Mixpanel/GA4 itself give funnel + anomaly + segment-contribution analysis out of the box; a 200-line scheduled notebook does the mix/rate decomposition and emails a brief. Bible Â§14.6 even admits a "Mapping to dbt semantic layer / MetricFlow" exists. The *only* genuinely custom asset is `decompose_change` (commodity algebra) and the eval harness. Helios chooses to hand-build a 5-server MCP fabric, a 7-agent SDK orchestration, a bespoke memory+vector store, and a CI eval â€" for a single static 3-month public dataset. The build cost vastly exceeds the buy-or-notebook cost, and the differentiator (Simpson's-paradox-aware decomposition) is a feature, not a platform. **Fundamental.** The defensible core is one function; the rest is reinvented infra.

[HIGH] One-person maintenance surface is unsustainable and self-contradicted by the "production / multi-tenant" roadmap

Claim attacked. Solo portfolio cadence (DEVELOPMENT_PLAN.md Â§5: "solo portfolio") maintaining dbt project + 16-dim semantic registry + 5 MCP servers + 7 agents + memory DDL (8 tables, Bible Â§22) + vector store + eval harness (injector + 50 scenarios + 6 scorers + gates) + CI â€" while the roadmap promises "Multi-tenant, real-time streaming, warehouse-agnostic" with "Snowflake/Databricks/DuckDB adapters" (Bible Â§23.4). **Why it fails.** The surface area is enormous: every component has its own contract, error taxonomy (14 error codes, MCP Â§5), config, and test suite, and they are tightly coupled (a registry column rename must stay in lockstep across the YAML, the marts, the resolver, the Finding envelope, and the eval labels â€" DEPENDENCY_MAP Â§8 "Continuity guardrails" is an admission of how fragile the coupling is). The roadmap then layers tenant isolation, streaming ingestion, and three warehouse adapters (Â§23.4) on top â€" for a dataset that is **static, historical, obfuscated, and 3 months long** (Bible Â§23.7). Promising warehouse-agnostic multi-tenancy when **zero lines run** (DEVELOPMENT_PLAN.md Â§2 "Code layer â€" not started") is roadmap fantasy that inflates the maintenance commitment without any evidence the single-tenant single-dataset core works. **Fundamental.** Scope outruns the maintainer and the data.

[HIGH] "Always-on / scheduled / autonomous" is architecturally incoherent against a frozen dataset

Claim attacked. "The product's heartbeat is the **autonomous scheduled run**" (CLAUDE.md Â§1); a scheduler entrypoint (A9.2) "fires" the Orchestrator on a cron (AGENT_ARCHITECTURE.md Â§10); Monitor does "anomaly detection over time" with `forecast` and seasonality. **Why it fails.** The dataset is "a fixed historical export (~2020-11-01 to 2021-01-31)" (Bible Â§20.1). Every scheduled run sees byte-identical data; nothing is "fresh"; there is nothing to monitor. The architecture commits a whole scheduling tier (DEPENDENCY_MAP A9.2, M11), a freshness SLA (Bible Â§15.5,

FR-A8 incremental "only new date shards"), a suppression list that decays over runs (PRIOR_HALF_LIFE_DAYS=60), and memory "loop closure" where "a later run sees seen-before" (AGENT_ARCHITECTURE.md Â§8) â€” all premised on a stream of new data that does not and will never exist here. The entire "proactive not reactive" pillar (CLAUDE.md Â§3.5) is unfalsifiable on the only data the system has. A scheduler firing the same FSM at the same frozen table is a cron job that recomputes a constant. **Fundamental.** "Always-on" has nothing to be on.

[MEDIUM] The "single source of truth" cannot agree on its own filename, schema keys, or scenario count

Claim attacked. "metric/dimension definitions live **only** in `models/semantic/semantic_models.yml`" (CLAUDE.md Â§5; MCP_ARCHITECTURE.md Â§3 config points `registry:` at `semantic_models.yml`); the resolver reads `m["sql"]`, `m["name"]`, `m["agg"]` (MCP_ARCHITECTURE.md Â§9, lines 292â€“313); "ships **34 scenarios**" (Bible Â§20.3). **Why it fails.** The file on disk is `models/semantic/semantic_layer.yaml` â€” wrong name in CLAUDE.md and MCP_ARCHITECTURE.md, so `semantic-mcp` as configured would fail to load its keystone. Its entries use `metric_name:` and `sql_definition:` (verified in the artifact), but `build_query` indexes `m["name"]` / `m["sql"]` / `m["agg"]` and the `Finding` envelope is built around `name` / `sql` â€” so the resolver cannot read the registry without an adapter that **does not exist** (the file's own header note admits "build_query maps metric_name->name, sql_definition->sql" â€” an unwritten translation layer). And `eval/scenarios/` contains **50** scenarios across 7 files, not the 34 cited everywhere (Bible Â§20.3, DEPENDENCY_MAP A10.3, DEVELOPMENT_PLAN.md Â§8/Â§12). The governance keystone â€” the thing the whole anti-hallucination guarantee rests on â€” has a filename drift, a schema-key mismatch requiring phantom glue, and a count that contradicts the spec. **Fixable.** Rename, write the adapter, recount â€” but it betrays the discipline.

[MEDIUM] "0 hallucinated columns" is a contract about the keystone, and the keystone is precisely where the artifacts already disagree

Claim attacked. "hallucinated columns are structurally impossible because the model can only reference governed metrics" â†' "0 hallucinated columns" (Bible Â§24.2, Â§4.5, NFR-T1). **Why it fails.** The guarantee is only as strong as the registry-resolver binding, and that binding is currently broken (prior finding): a resolver that can't read `metric_name` / `sql_definition` either crashes or requires hand-glue that reintroduces exactly the manual SQL the architecture swears it eliminates. Worse, the guarantee is about *columns*, not *reasoning*: governed SQL prevents naming a column that doesn't exist; it does nothing to stop Diagnose composing correct columns into a wrong causal story (a deploy, a price change, a competitor, a holiday â€” none observable in this data). The architecture markets structural impossibility of one narrow failure (bad column names) as if it were trustworthiness of the diagnosis. "0 hallucinated columns" is true and nearly worthless; the

failure mode that matters â€” hallucinated *causation over correct numbers* â€” has no structural guard at all. **Fundamental.** The guarantee guards the cheap failure, not the expensive one.

[MEDIUM] The Critic is a stochastic LLM grading another stochastic LLM with no ground truth â€” "adversarial verification" is not a guarantee

Claim attacked. "an adversarial Critic agent tries to refute every finding before it ships" as the third defensibility pillar (Bible Â§4.5); "Critic (Opus): adversarial reasoning that must out-think Diagnose" (AGENT_ARCHITECTURE.md Â§3). **Why it fails.** Critic and Diagnose are the *same model family* (both Opus) with correlated blind spots; a confound Diagnose misses, Critic is statistically likely to miss too (shared training, shared priors). The Critic has no oracle â€” its `recall_prior` /seasonality checks are only as good as a hand-seeded `seasonality_calendar` (DEPENDENCY_MAP Â§8 warns an "unseeded seasonality_calendar makes the Critic blind"), and its verdicts (PASS/DOWNGRADE/DROP) are sampled tokens, not proofs. Architecturally this adds an entire Opus stage, a bounded re-query loop, and a suppression-list side-channel to obtain a *probabilistic* second opinion that the architecture then presents as the trust pillar. Stacking two samplers does not manufacture a guarantee; it manufactures cost and the *appearance* of rigor while leaving the actual correctness ungrounded (it rests entirely on the offline eval â€” which grades arithmetic the system itself defines as truth). **Fundamental.** Verification by a peer sampler is not verification.

Principal Analytics Engineer Review

Headline kill. The entire "85% root-cause accuracy" proof is a closed loop: the eval injector perturbs *aggregates* and then computes the "true" mix/rate/interaction split "by **the same decomposition algebra Helios is graded on**" (Bible Â§20.2, step 3), so the benchmark grades Helios's arithmetic against Helios's own arithmetic applied to data Helios itself constructed. That is a unit test of an algebraic identity (`Î”R = mix + rate + interaction` , which is *true by definition*), dressed up as an empirical accuracy claim. Worse, the input to that decomposition â€” `fct_funnel_by_dim` â€” is documented to hold exactly **one dimension at a time** (DATA_MODEL.md Â§5.10), yet 6 of the 50 labeled scenarios target two-dimensional cells like `Paid Search Ã— mobile` (scenarios.yaml). The mart physically cannot represent the segment the label says is the root cause. The headline number is unearned twice over.

[CRITICAL] The decomposition input mart cannot represent the segments the eval grades against

Claim attacked. `fct_funnel_by_dim` is "the funnel rolled up by a single **canonical dimension** … one canonical dimension at a time" with PK `(date_key, dimension, dimension_value)` , and it

is "the direct input to the mix-vs-rate decomposition (`decompose_change`)" (DATA_MODEL.md Â§5.10, Â§2.1 catalog). Bible Â§20.2 says the decomposition gold target is computed on the `fct_daily_funnel` grain. **Why it fails.** `decompose_change` requires a `(w_i, r_i)` vector over the *segments of one partition of the population*. Six labeled scenarios (S001 and five siblings) declare `root_cause_segment: {channel_group: "Paid Search", device_category: "mobile"}` â€" a cross-tab cell. `fct_funnel_by_dim` long-unpivots to one dimension, so it has a `channel_group=Paid Search` row and a `device_category=mobile` row but **no `Paid Search Ã– mobile` row**; their session counts overlap and cannot be summed or differenced into a single weight. To grade these scenarios you must decompose over the `channel_group Ã– device_category` cross product, which only `fct_daily_funnel` carries (and only for its four hardcoded dims). So the two decomposition feeds disagree on grain, and the single-dimension mart the doc names as "the direct input" is the wrong table for 12% of the benchmark. This is a design contradiction, not a typo. **Fundamental.** The decomposition input grain contradicts the labels.

[CRITICAL] "Diagnose WHY" is arithmetic attribution, not causal inference

Claim attacked. The thesis: an engine that "diagnoses **why** an e-commerce funnel moved" (CLAUDE.md Â§1); the decomposition "dissolves Simpson's paradox" and is "the technical centerpiece" (Bible Â§19, Â§4). **Why it fails.** `Î”R = Î£ Î”w_iÂ·r_i(t0) + Î£ w_i(t0)Â·Î”r_i + Î£ Î”w_iÂ·Î”r_i` is a tautological re-expression of an aggregate change â€" it tells you *where in the segment grid* the number moved and whether the mover was composition or in-segment rate. It says nothing about *cause*: a deploy, a price change, a competitor, a promo, a holiday, a tracking break. The mart layer has no event log, no deploy calendar, no price-change feed except a single SCD2 snapshot on item price (`snap_dim_items`), and explicitly **no cost data** (DATA_MODEL.md Â§5.3). So the strongest honest claim is "the rate of checkout-to-purchase fell *in the Paid Search mobile cell*," which the system then narrates as a *why*. Relabeling "where" as "why" is the central overclaim, and grounding the SQL does nothing to prevent the wrong causal story over correct numbers. **Fundamental.** Attribution algebra is not causation.

[HIGH] The whole channel decomposition rests on session-scoped source/medium that GA4's obfuscated sample barely populates

Claim attacked. "Prefer the session-scoped `event_params.source` / `medium` ... fall back to the user-level `traffic_source` struct only when the session params are NULL" (DATA_MODEL.md Â§5.3); channel is "the precondition for honest mix-vs-rate decomposition" (Â§1.3). **Why it fails.** In the `ga4_obfuscated_sample_ecommerce` export, per-event `event_params.source` / `medium` are sparse and frequently absent outside the acquisition event; `traffic_source.*` (user first-touch) is the field that is reliably populated. The doc itself warns first-touch fallback "mis-attributes every returning session to its original acquisition channel" (Â§5.3) â€" yet the coalesce makes first-touch

the fallback precisely when session scope is NULL, which on this dataset is *most non-landing events*. So `channel_group`, the headline decomposition dimension, silently degrades toward first-touch for a large share of sessions, and `int_ga4__sessionized` resolves it with `max(event_source)` / `max(event_medium)` (DATA_MODEL.md Â§5.5 SQL) â€" a non-deterministic pick of "a" source within the session, not the earliest. Two different sources in one session resolve by alphabetical MAX, not by first-touch order. The channel axis the entire mix-vs-rate story stands on is itself a noisy, partially-first-touch, order-insensitive guess. **Fundamental.** The dataset cannot supply clean session-scoped channel.

[HIGH] The cohort SQL is broken and the retention numerators are duplicated across ages

Claim attacked. `fct_cohorts` "pre-computing `cohort_size` and `retained_users_d1/d7/d30`" so retention metrics "resolve as simple `SUM(retained_users_dN)/SUM(cohort_size)`" (DATA_MODEL.md Â§6.3, with copy-usable SQL). **Why it fails.** The shipped query does not compile and would not mean what it claims if it did. It writes `cross join unnest([1,7,30]) as age_days with offset` but then selects `ages.age_days` and groups by `ages.age_days` â€" the table alias is `age_days`, there is no `ages`, and `with offset` introduces a second unnamed column; this is a reference error. Worse, the three numerators are `date_diff( ... ) between 1 and 1`, `between 1 and 7`, `between 1 and 30` â€" *hardcoded constants that ignore the `age_days` axis entirely*. So every one of the three age rows for a cohort carries the **same** d1/d7/d30 triple; the `age_days` grain is decorative. And the `returns` CTE computes a per-user `min( ... ) over (partition by user_pseudo_id) as ignore_me` window it never uses, then joins on `user_key` against a fan-out of every session date. This is not "production-grade" SQL (Â§Purpose); it is pseudocode that was never run â€" consistent with the project's own status: no code built yet. **Fixable.** Rewrite the snippet â€" but it discredits "production-grade."

[HIGH] day_30 retention is mostly unobservable on a 3-month frozen sample â€" the metric is theater

Claim attacked. `day_30_retention` is a first-class semantic metric (semantic_layer.yaml) and a named eval grain; the doc concedes "the ~3-month window right-censors `day_30` for late cohorts" and says it "must be restricted to *mature* cohorts" (DATA_MODEL.md Â§3 catalog, Â§6.3). **Why it fails.** The window is 2020-11-01 to 2021-01-31, 92 days. A cohort needs â‰¥30 days of forward observation for an *uncensored* d30, so only acquisition weeks in roughly Nov 1â€"Jan 1 qualify, and the *useful, mature* cohorts collapse to a handful of early weeks â€" exactly the weeks with the least traffic at the dataset's leading edge. After the "restrict to mature cohorts + minimum cohort_size" filters the doc mandates, `day_30_retention` is computed from a tiny, non-representative slice. Shipping it as a governed headline metric on a static historical export is freshness/maturation theater: it can never mature further because the data never advances. The

metric exists to look complete, not to be trustworthy. **Fundamental.** A frozen 92-day window cannot support d30 cohorts.

[HIGH] Source-freshness and the autonomous schedule are pure ceremony on a static export

Claim attacked. "`dbt source freshness` runs first and its exit code gates everything downstream ... A stale source must block diagnosis" (DBT_GUIDE.md Â§2); the product's "heartbeat is the autonomous scheduled run" (CLAUDE.md Â§1). **Why it fails.** DBT_GUIDE.md Â§2 then admits the sample is static, the newest shard is `events_20210131`, "Real freshness against 'now' (2026) is therefore meaningless: every freshness check would scream `error`," so the gate is disabled on the sample and `dbt_utils.recency` is downgraded to `warn`. So on the only data that exists, the freshness machinery is wired to do nothing. Every "scheduled run" sees the identical frozen 92 days; there is no new shard to detect, no anomaly that wasn't there yesterday, no day_30 that matured, nothing to monitor. "Always-on autonomous diagnosis" over a static table is a cron job that recomputes the same answer forever. The freshness contract is "real, production-grade, and fully implemented" only in the sense that it is correctly configured to be inert. **Fundamental.** Nothing live exists to monitor or refresh.

[MEDIUM] The 47-metric semantic layer is bloated with metrics this dataset cannot honestly support

Claim attacked. "47 governed metrics" as a feature (DATA_MODEL.md Â§1, Â§1.4); 13 are category `supporting` and 4 are `acquisition` (semantic_layer.yaml category counts). **Why it fails.** `cac_proxy` is labeled "CAC Proxy (**NOT** true CAC)" and its own caveats say "the GA4 export has **no** cost/spend column, so a dollar CAC is impossible here" â€” it is `paid_sessions/new_purchasers`, a "unitless ratio" with a five-bullet do-not-use list. A metric that requires four paragraphs explaining what it *isn't* is liability, not asset. The product cluster (`product_view_rate`, `product_conversion_rate`, `item_view_sessions`, etc.) depends on item-level `view_item` array tracking that the doc itself says cannot be linked to purchases within a session or even a cookie (DATA_MODEL.md Â§7.5), so "product conversion" is flagged as a cross-session approximation the Critic must caveat â€” i.e. it isn't a conversion rate. Thirteen `supporting` measures (`retained_users_d1/d7/d30`, `item_views`, `orders_with_item`, `paid_sessions`...) are mostly denominators/numerators of the headline metrics re-exposed as first-class metrics, inflating the count. The 47 is a surface-area number; the load-bearing set is perhaps 15. **Fixable.** Cut the proxies and the duplicate components.

[MEDIUM] "Reconcile to the cent" and "0 dangling refs" are asserted, not earned â€” and partly self-contradicting

Claim attacked. Revenue must "match the source to the cent (`revenue_reconciles` test)" (CLAUDE.md Â§8 keystone 4); `SUM(session_revenue)` over `fct_funnel` "equals `SUM(gross_revenue)` over `fct_orders` at the grand total (within the 0.5% `reconcile` tolerance)" (DATA_MODEL.md Â§8.4); the YAML "passed a referential-integrity LINT." **Why it fails.** "To the cent" and "within 0.5%" are not the same claim, and they appear in the same project as co-equal guarantees. More importantly, the two revenue paths are not built the same way: `fct_orders` derives `gross_revenue` from `ANY_VALUE(ecommerce.purchase_revenue_in_usd)` after grouping on `(order_key, session_key, user_pseudo_id, order_ts)` (DATA_MODEL.md Â§8.2 SQL) â€" but if a `transaction_id` 's duplicate rows carry *different* timestamps, that GROUP BY produces **two rows for one transaction**, defeating the dedup the section is named after. Meanwhile `session_revenue` dedups on `transaction_id` within the session. A purchase whose duplicate rows split across two `ga_session_id` s (e.g. a midnight-crossing session, which the doc says splits, Â§5.1) lands in two `fct_funnel` rows but one `fct_orders` row â€" so the grand totals are *not guaranteed* equal even in principle. The reconciliation is asserted as an invariant; the dedup logic shown does not establish it. And a YAML lint proves names resolve, not that any number reconciles to anything â€" no query has run. **Fixable.** Fix the dedup keys; but the "to the cent" claim is unproven.

[MEDIUM] The wide-fact assumption contradicts the documented marts the decomposition needs

Claim attacked. "Wide facts ... descriptive dimensions (`device_category` , `country` , `channel_group` , `is_new_user` , `landing_page`) are denormalized directly onto `fct_funnel` " so the layer "slices a metric by simply `GROUP BY` -ing a column on the same fact, with no join" (DATA_MODEL.md Â§1.5). **Why it fails.** The `fct_funnel` column list in Â§5.8 carries only `device_category` , `country` , `is_new_user` (plus keys and `reached_*`). It does **not** carry `operating_system` , `browser` , `region` , `source` , `medium` , `campaign` , `landing_page` , or `session_number_bucket` â€" yet the `revenue` , `sessions` , `users` , and `revenue_per_session` metrics in semantic_layer.yaml all advertise `dimensions_supported` including `operating_system, browser, region, source, medium, campaign, landing_page, session_number_bucket` . So the semantic layer promises to slice `revenue` by `landing_page` on a fact that does not contain `landing_page` . Either the "no join at query time" property is false (you must join back to `fct_sessions` , which *does* carry those dims) or the fact must be widened far beyond what Â§5.8 documents. The single-source-of-truth catalog and the registry disagree on what columns exist â€" and the agents will request dims that aren't there. **Fixable.** Widen the fact or document the joins â€" but as written they conflict.

[MEDIUM] The single-source-of-truth discipline is already violated in the canonical docs

Claim attacked. "metric/dimension definitions live **only** in `models/semantic/semantic_models.yml`" and "`semantic-mcp`'s `registry:` ... must point at that exact path" (CLAUDE.md Â§5, Â§8 keystone 2); the Bible's Reference Card "wins over any prose anywhere" (CLAUDE.md Â§2). **Why it fails.** The actual registry is `models/semantic/semantic_layer.yaml` (v2, 47 metrics), whose header explicitly "Supersedes `semantic_models.yml` (v1)." Yet CLAUDE.md Â§5 *and* Â§8, plus MCP_ARCHITECTURE.md Â§lines 57/74/159, all still name `semantic_models.yml` as the keystone and the registry path. The playbook itself logs this as a live bug ("Filename drift â€" `mcp_servers.yaml` and `hallucination.py` must point at `semantic_layer.yaml` not `semantic_models.yml`; a stale path makes the AST check pass everything," 05_m10_m12.md). So the document that defines the *single source of truth rule* points the hallucination gate at a file that doesn't exist â€" meaning the "0 hallucinated columns" check would load nothing and pass everything. Separately, the registry's own resolver contract â€" "maps `metric_name` â†' `name`, `sql_definition` â†' `sql`, `aggregation_method` â†' `agg`" (semantic_layer.yaml header) â€" is an adapter the schema describes but no code implements; `build_query` per the Bible expects `name` / `sql`. A project whose three "resume-point" docs disagree about the name and shape of its governance keystone has not earned the word "governed." **Fixable.** Rename and align â€" but the discipline it claims is already broken.

[MEDIUM] `engaged_session` and source resolution use `MAX` / `ANY_VALUE` as if attributes were stable, silently corrupting slices

Claim attacked. Device/geo are "`any_value` within the session (stable per visit)"; engagement is `max( ... )`, source is `coalesce(max(event_source), max(first_touch_source))` (DATA_MODEL.md Â§5.2, Â§5.5 SQL). **Why it fails.** `ANY_VALUE` and `MAX` are not "the session's value" â€" they are *whatever the engine returns*, and the doc conflates the two. `MAX(event_source)` over a session with both `google` and `(direct)` returns `google` by string ordering, not by recency or first-touch; `MAX(device_category)` over a session that GA4 logged as both `mobile` and `tablet` (device switches, appâ†'web) silently picks `tablet`. These columns are the *exact* slicing keys the decomposition partitions on. When the eval injects a clean `rate_multiplier: 0.55` into the `Paid Search Ã– mobile` cell, but real sessionization smears a fraction of those sessions into `tablet` or `(direct)` via MAX, the injected ground-truth cell and the reconstructed cell don't line up â€" and the MAPE/accuracy the system reports against its own labels degrades for reasons that have nothing to do with diagnosis quality. The keystone the doc says "fails silently" indeed does, by its own aggregation choices. **Fixable.** Use ordered `ARRAY_AGG ... LIMIT 1` consistently â€" but as written the spine is non-deterministic.

Principal Product Analyst Review

Headline kill. Helios sells "diagnose *why* the funnel moved" but its engine only computes *where* the arithmetic moved (which segment, mix vs. rate). The actual "why" â€” a deploy, a price change, a broken SDK, a competitor, a holiday â€” lives entirely outside the dimensional space Helios can query, yet the product's value prop, its benchmark labels (`expected_diagnosis`), and its sales pitch all assert that causal layer anyway. The system's own scenario file proves this: in S021 the injector does nothing but multiply a rate by 0.62, but the label "expected_diagnosis" claims a "payment SDK regression on iOS WebView" â€” a fact found nowhere in the data and impossible to derive. Helios is an attribution calculator wearing a diagnosis costume, and the costume is the entire business case.

[CRITICAL] "Diagnose WHY" is an unbridgeable overclaim â€” the engine produces WHERE

Claim attacked. Bible Â§1.1/Â§4.1: "continuously diagnoses *why* an e-commerce funnel is moving"; Â§2.4: dashboards "report *what*, never *why*â€¦ Helios provides causal-style attribution." Â§1.2 names the category "autonomous growth diagnosis" whose promise is "causal-style attribution of metric movement."

Why it fails. The technical centerpiece (Â§2.3 decomposition) answers a strictly *positional* question: which segment's weight or rate moved, and by how much. That is WHERE, not WHY. The causes a Head of Growth actually needs â€” "you shipped a checkout regression," "Safari broke third-party cookies," "a competitor undercut price," "it's just January" â€” are not columns in `fct_daily_funnel` ; they are exogenous events. Helios literally cannot observe them. The Bible silently smuggles the causal layer in via narrative: the scenario labels (scenarios.yaml S021 "payment SDK regression," S023 "PDP price-display change," S024 "shipping-calculator failure," S026 "forced account-creation gate") are the *injector's* fictional cover story, never something the pipeline could recover from a rate multiplier. Grading the LLM against those strings rewards plausible storytelling, not diagnosis. "Causal-style" is the tell: it is the adjective you use when you cannot say "causal."

Fundamental. The data lacks cause columns.

[CRITICAL] The autonomous / always-on value prop is undemonstrable on a frozen 3-month dataset

Claim attacked. Â§1.1: "an always-on AI Growth Analystâ€¦ diagnosis becomes a continuous, autonomous, proactive background process. Every scheduled run, Helios re-derives the fullâ€¦ funnel." Â§1.4 vision: "before a human has to ask." Â§4.4 anti-product: "runs on a schedule and reports without being asked."

Why it fails. The entire wedge is *autonomy over a live funnel*, and there is no live funnel. The dataset is static, historical, obfuscated, ~2020-11-01 to 2021-01-31 (Bible Â§20.1 admits "a fixed historical export"). Every "scheduled run" sees byte-identical data; a cron job re-deriving the same frozen 92 days produces the same brief forever. There is no "freshness," no new anomaly to catch "before a human asks," no decision to be made "in <5 minutes." The product's heartbeat â€" the proactive scheduled run (Principle 5) â€" has nothing to beat against. The only anomalies Helios will ever "catch" are the ones the eval harness *injects into a clone* (Â§20.2). So the headline differentiator versus Amplitude/Mixpanel ("autonomous, not pull") is demonstrated exclusively on synthetic perturbations the system planted itself. There is no artifact in this repo that can show the autonomous loop delivering value on real movement.

Fundamental. Static data cannot exhibit autonomy value.

[CRITICAL] revenue-at-risk is a counterfactual resting on a false persistence assumption, then sold as a hard dollar number

Claim attacked. Glossary "Revenue-at-risk": "dollars recoverable if a degraded rate returned to its t0 / forecast baseline (â‰ˆ Î"rate Ã— affected sessions Ã— downstream value)." Â§4.7: every finding "carriesâ€¦ a dollar revenue-at-risk." Â§5.4 lists "Dollar at-risk surfaced" as a business metric.

Why it fails. The formula assumes the t0 rate *would have persisted* absent the change â€" a counterfactual that is routinely false in e-commerce (post-holiday troughs, promo expiry, seasonal demand, mean reversion). The Bible knows this is fragile: Â§20.1 calls the labels "a *counterfactual*," and the dollar basis in scenarios.yaml (e.g. S001 "$38,000â€¦ (rate_t0 â^' rate_t1) Ã— begin_checkout_sessions_t1 Ã— aov") is computed *only because the injector knows the unperturbed truth.* On real data there is no unperturbed twin, so "revenue-at-risk" becomes "Î"rate Ã— volume Ã× AOV holding everything else fixed" â€" a back-of-envelope figure dressed as a measured liability. Presenting `$72,000` (S024) to a Head of Growth as if it were a reconciled number, when it is the product of an unfalsifiable persistence assumption, is the kind of false precision that destroys analyst trust the first time the "recovered" dollars never appear.

Fundamental. No counterfactual is observable in production.

[HIGH] The 85%-vs-45% benchmark is a unit test of the system's own arithmetic, not a measure of diagnostic skill

Claim attacked. Â§5.6/Â§20: "root-cause accuracy â‰¥85% (vs â‰¤45% naive baseline)" â€" "the project's central empirical claim" (Â§20.5). Â§24.1 pitch and Â§25.2 resume both lead with

"85%+â€¦ vs 45%."

Why it fails. The injector perturbs aggregates using the *same* mix/rate/interaction algebra Decompose uses to diagnose (Â§20.2: "the *true* mix/rate/interaction contributions computed analytically by the same decomposition algebra Helios is graded on"). The "ground truth" and the "diagnosis" are the same equation run forward then backward. Scoring 85% on this is not evidence of analytical accuracy; it is a check that the arithmetic is implemented without bugs â€” a property a 20-line pandas function would also have. Worse, the â‰¤45% baseline ("largest absolute segment delta," Â§20.5) is a deliberately chosen strawman: no competent analyst declares a root cause from a single absolute delta without normalizing by rate. The "near-doubling" headline (Â§20.5) is the distance between a correct calculator and a calculator someone broke on purpose. As a *product* claim ("85% accurate diagnosis"), this number means almost nothing about whether real briefs are correct or trusted.

Fundamental. Ground truth is defined by the system under test.

[HIGH] Mix-vs-rate decomposition does not deserve to be THE centerpiece â€” it is one routine technique every analyst owns

Claim attacked. Â§4.2: "The wedge is one painful, high-frequency, expensive jobâ€¦ automated mix-vs-rate root-cause diagnosis." Â§2.3 / Glossary: the decomposition is "Helios's technical centerpiece." Â§24.6 whiteboard: `Î”R = mix + rate + interaction`.

Why it fails. Mix/rate (a.k.a. attribution analysis, contribution decomposition, the "shift-share" method) is standard analyst toolkit â€” a GROUP BY plus a weighted-average identity. It is taught, scripted, and shipped inside existing tools (Amplitude/Mixpanel segmentation, GA4 "Insights" anomaly + contribution, any analyst's notebook). Building an entire seven-agent, five-MCP-server, 25-section system around one well-known identity is a category error: it elevates a *technique* to a *product*. The Bible itself concedes the technique is insufficient â€” Â§23.5 defers "true causal inference," "uplift modeling," and "double-ML" to Phase 4 with the admission that P1â€“P2 ship only "correlational decomposition." So the centerpiece is explicitly the *weakest* form of the analysis, and the genuinely hard parts (causality) are roadmap. A Head of Growth will not adopt a new "category" for a calculation their analyst already does in an afternoon.

Fundamental. A known identity cannot anchor a new category.

[HIGH] The "insight-to-action gap" is misdiagnosed â€” Helios automates RCA but not the decide/prioritize/ship that actually gates action

Claim attacked. Â§2.1: "The gap between 'we see a number moved' and 'we changed the business' is the single most expensive inefficiency in growth analytics. Helios targets that gap directly." Â§1.6: "demonstrably shortens the insight-to-action loop from weeks to a single review cycle."

Why it fails. The insight-to-action gap is rarely bottlenecked on *generating* the RCA. It is bottlenecked on organizational decisioning: getting eng capacity allocated, prioritizing against the roadmap, securing stakeholder buy-in, and actually *shipping* the fix. Helios does none of that â€” it stops at a Decision Brief and an experiment it cannot run (Â§21 intro: "there is no live traffic to A/B test against"). It hands a PDF to the same humans who were already the bottleneck, now with more findings to triage. The Â§1.3 "AFTER" diagram ends at "decision made in <5 minutes of human reading time" â€” but reading is not deciding, and a brief is not a shipped change. By over-producing diagnoses (every scheduled run, every metric, every segment) Helios risks *widening* the action gap with alert volume, which is precisely why it needs a suppression list (FR-B4) â€” a band-aid for a firehose the product itself creates.

Fundamental. The named bottleneck is downstream of what Helios builds.

[HIGH] The ROI story ("days to minutes") assumes RCA volume is the cost driver; it isn't

Claim attacked. Â§2.2: manual RCA is "~1â€“3 analyst-days" per anomaly, "the exact work Helios automates to a <5 min/run autonomous process." Â§5.2 baseline "~1â€“3 analyst-days (manual)" â†’ target "<5 min/run."

Why it fails. This is a table of unsourced, self-serving estimates ("typical effort," "frequently never done") engineered to make the savings look enormous. Two problems. (1) The "1â€“3 days" is per *deep* RCA; most metric wiggles need a 10-minute glance, not 3 days, so multiplying by "metrics Ã— segments Ã— weeks" (Â§2.2) inflates a denominator that doesn't exist. (2) Even granting the saving, analyst *RCA hours* are not the dominant cost in a growth org â€” eng time on the fix, experiment runtime, and opportunity cost dwarf it. Automating the cheap step (find the segment) while leaving the expensive steps (build, test, ship) untouched is classic ROI theater. And the "<5 min/run" target is itself unmeasured (status: no code built), so the entire before/after comparison is target-vs-target.

Fixable. Re-scope ROI to measured cost drivers.

[HIGH] The experiment-design framework prescribes tests the dataset can never run, and proves its own backlog is mostly impractical

Claim attacked. Â§4.1/Â§21: "prescribes a prioritized, statistically-defensible experiment backlog." Â§3.2 Marcus: "design the experiment." Every scenario's `expected_recommendation` ends "Size via power_analysis."

Why it fails. Â§21 admits the data is "OBSERVATIONAL and historical â€" there is no live traffic to A/B test against," then the Â§21.2 worked example computes that the flagship card H-2021-0042 needs ~14,800 sessions/arm against ~95/day â† **311 days â† ' UNDERPOWERED**. The product's signature output (a powered experiment) is, by its own math, un-runnable at the segment grain where its diagnoses live. The fallback (Â§21.2: "roll up to a coarser segmentâ€¦ or relax mde") destroys the very segment-precision the decomposition spent seven agents establishing. So Prescribe ships hypothesis cards (scenarios.yaml: "A/B test a sticky mobile add-to-cart button," "expedited guest-checkout") that no one can execute on this data, sized for a funnel that no longer receives traffic. A backlog of un-runnable experiments "ranked by money" (Â§21.4 ICE) is a vanity artifact, and the quasi-experimental DiD fallback (Â§21.5) is circular here because the only "treatment" is the injection the harness applied.

Fundamental. Observational static data forecloses experimentation.

[MEDIUM] Persona realism: no Head of Growth overrides their analyst on an autonomous AI's dollar figure

Claim attacked. Â§3.1 Priya: "Success criteria: Time-to-diagnosis <5 min reading; â‰¥85% of root causes correct." Â§24.3(a) STAR: "the decomposition gave an auditable answer that overrode the gut call on checkout." Â§3.1 anti-needs: "does NOT wantâ€¦ a chatbot to interrogate."

Why it fails. The persona is constructed to want exactly what Helios outputs and to reject exactly what it doesn't build (a chatbot, raw tables). That is reverse-engineered, not observed. In reality a Head of Growth who is "chronically time-poor" and "reports to the exec team weekly" will *not* forward a CEO a `$72,000 revenue-at-risk` from an unsupervised LLM without her analyst sanity-checking it first â€" which reintroduces the human RCA loop the product claims to remove, and means the analyst (Dana) must now *audit Helios* on top of her old job. The Â§3.1 claim that Priya wants no interrogation surface contradicts trust-building: the first time a brief is wrong (and on real exogenous causes it will be â€" see Critical #1), she will demand to drill in, i.e. demand the chatbot the anti-product (Â§4.4) forbids. The personas assume trust that the product has not earned and structurally cannot earn on causal claims.

Fixable. Personas can be re-grounded in real buying behavior.

[MEDIUM] Category/moat claim ("creates a new category") ignores shipped incumbents doing the same job

Claim attacked. Â§1.2: "It creates a new category: **autonomous growth diagnosis**." Â§2.5: "None of them autonomously decompose mix-vs-rate, price the movement, and ship a verified brief on a schedule. That white spaceâ€¦ is precisely Helios's category."

Why it fails. The "white space" is asserted, not evidenced, and is largely already occupied. GA4 itself ships automated anomaly detection with contribution analysis and emailed Insights *on a schedule*; Amplitude and Mixpanel ship anomaly alerts, automated root-cause / "Compass" style contribution, and AI summaries. The Bible's own comparison table (Â§1.2) concedes Amplitude/Mixpanel answer "What, with some who" and have "recent 'AI' features" (Â§2.5) â€” then waves them away as "still fundamentally descriptive." But pricing the movement in dollars and emailing a segmented contribution breakdown is not a new *category*; it is a feature delta over incumbents who have distribution, integrations, and live data Helios lacks. "Creating a category" is a resume/interview framing (Â§24, Â§25) not a defensible moat â€” there is no proprietary data, no network effect, and the core technique is public-domain arithmetic.

Fixable. Reframe as a feature, drop the category claim.

[MEDIUM] The success metrics are dominated by vanity/leading proxies, not decision-grade outcomes

Claim attacked. Â§5.2 lists seven "product-quality" metrics, all "Leading"; the North Star (Â§5.1) is "Verified, actioned Decision Briefs that correctly diagnose root cause," but the *measurable* lagging outcomes (Â§5.4: "Decisions influenced," "Misdiagnosis cost avoided," "Experiments shipped") are exactly the ones the static dataset makes unobservable.

Why it fails. "0 hallucinated columns," "100% governed SQL," "100% findings carry significance + $," "scheduled-run completion rate" (Â§5.3) are all *process compliance* metrics â€” they measure that the plumbing ran, not that any decision improved. They are trivially satisfiable (a system that always emits the same governed query and the same dollar formula scores 100% on every one) and tell a buyer nothing about value. Meanwhile the only metrics that would prove the thesis â€” decisions influenced, misdiagnosis cost avoided, experiments shipped and won (Â§5.4, all "Lagging") â€” are uninstrumentable on a frozen 3-month sample with no users and no action-tracking signal. So the dashboard of "success" is entirely the vanity column, and the North Star's "actioned" and "influence a decision" clauses are, on this data, permanently unmeasurable. A product whose only measurable metrics are the ones that don't matter is optimizing the wrong loop.

Fixable. Promote decision-grade outcomes; demote compliance proxies.

Principal AI Engineer Review

Headline kill. The 85%-vs-45% headline is not a measurement of diagnostic accuracy â€” it is a tautology dressed as a benchmark. The eval injects anomalies by perturbing aggregates with the *exact* mix/rate/interaction algebra that `Decompose` uses to diagnose (Bible Â§20.2: ground-truth labels are "the *true* mix/rate/interaction contributions computed analytically by the same decomposition algebra Helios is graded on"), so on the injected data the decomposition is correct *by construction* â€” it is arithmetically impossible for it to be otherwise. The benchmark therefore grades whether the FSM can run the same equation twice and whether an Opus model can read off the largest term. That is a unit test of `decompose_change`, not evidence that Helios diagnoses real funnels. And even this circular number is a TARGET printed in a results table (Â§20.9) with no code behind it.

[CRITICAL] The benchmark is circular: ground truth is generated by the system under test

Claim attacked. "root-cause segment accuracy **>= 85%** on the labeled benchmark vs **<= 45%** for the naive baseline" (Bible Â§20, repeated in CLAUDE.md Â§4 and AGENT_ARCHITECTURE.md Â§13 exit gate). The injector "applies one of two perturbation primitivesâ€¦ changes `r_i` only -> ground-truth = **rate-change**" and records "the *true* mix/rate/interaction contributions computed analytically by the same decomposition algebra Helios is graded on" (Â§20.2). MCP_ARCHITECTURE.md Â§6.3 confirms `decompose_change` "implements the FOUNDATION identity exactly."

Why it fails. The injector multiplies one cell's rate by `rate_multiplier` while holding `sessions` fixed (Â§20.2 step 2). Run the identity `Î”R = Î£ Î”wÂ·r0 + Î£ w0Â·Î”r + Î£ Î”wÂ·Î”r` on that exact mutation and you get, definitionally, `mix=0, rateâ‰ 0, interaction=0` â€” which is precisely what every `single_segment_rate` scenario's `mix_rate_interaction: {mix: 0.000, rate: −0.013, interaction: 0.000}` records (scenarios.yaml S001 et seq.). The "gold target" and the model's answer are computed by the *same function on the same conserved aggregates*. The decomposition MAPE â‰¤10% target (Â§20.4) cannot fail unless the code has a literal bug; Â§20.2 even admits "the analytic mix/rate/interaction split is exact." This proves the arithmetic is self-consistent. It proves nothing about whether the system can diagnose an anomaly it did not itself author. A passing benchmark here is compatible with 0% real-world accuracy.

Fundamental. Eval and system share one algebra.

[CRITICAL] Injected anomalies do not resemble real anomalies â€” only the ones the algebra can see

Claim attacked. "we inject synthetic-but-known anomaliesâ€¦ The pipeline must rediscover what we hid" and the 7-bucket taxonomy (Â§20.3) presented as the proof the product "diagnoses WHY an e-commerce funnel moved" (thesis, Â§4).

Why it fails. Every injected scenario lives *inside the dimensional space the tree searches*: a clean rate multiplier on one or two `(channel_group, device_category, country, browser, â€¦)` cells (scenarios.yaml S001â€“S032). Real funnel anomalies are the opposite: (1) caused by factors *outside* any GA4 dimension â€” a deploy at 14:00, a price change, a competitor promo, a payment-processor outage â€” which the dimensional tree can never name; (2) the product of many overlapping causes with no single dominant cell; (3) instrumentation drift that corrupts the very numbers being decomposed. The benchmark never tests case (1) at all, and the `multi_segment_mixed` bucket is only 6 scenarios of perfectly factored mix+rate (S027â€“S032). So the eval validates exactly the regime where best-first-by-rate-effect (Â§19.4) is guaranteed to win and is silent on every regime where it fails. "Rediscover what we hid" is the tell: the system is graded only on findable injections, never on the unfindable causes that dominate real diagnosis.

Fundamental. Real causes live outside the dimension set.

[CRITICAL] "Diagnose WHY" is an overclaim â€” the engine does arithmetic attribution, not causal inference

Claim attacked. The thesis that Helios "diagnoses WHY an e-commerce funnel moved" (Â§4) and the Diagnose prompt: "Find the root cause by best-first search" (AGENT_ARCHITECTURE.md Â§6.4).

Why it fails. `decompose_change` and the hypothesis tree answer *where* the number moved (which cell, mix vs rate), not *why*. Yet the expected diagnoses invent mechanisms wholesale: S021 asserts "a payment SDK regression on iOS WebView," S024 "a shipping-cost estimator failure," S023 "a PDP price-display/strikethrough change." Nothing in the data supports these stories â€” the injector only multiplied a rate. The mechanism is a confident LLM confabulation layered over a correct number. Grounding (Â§19.2 G1â€“G5) guarantees the *columns* are real; it does nothing to make the *causal narrative* real. The benchmark's `expected_diagnosis` strings actually *reward* this confabulation by writing the invented mechanism into the answer key, training/grading the system to assert causes it cannot observe.

Fundamental. Attribution â‰ causation; data lacks the cause.

[CRITICAL] The Critic is a stochastic LLM with no ground truth â€” "adversarial verification" is not verification

Claim attacked. Trust pillar 3: "adversarial verificationâ€¦ the Critic agent refutes every finding." Critic prompt: "Return PASS only if every refutation fails" (AGENT_ARCHITECTURE.md Â§6.5); the

Critic is Opus, the FSM routes every finding through it (Â§19.3).

Why it fails. The Critic is another temperature-0-ish Opus sampler with no oracle. It cannot *verify* anything; it can only generate a second opinion about the first opinion. There is no ground truth at runtime â€” the labels exist only in the eval harness, never in production. "Default to skepticism" is a prompt instruction, not a guarantee; an LLM that hallucinated a plausible iOS-SDK story (Finding) is exactly the LLM most likely to find that story plausible (Critic), because both share training priors and see the same evidence bundle. Worse, the Critic's four refutation axes (confound/sample/seasonality/data-quality) are the same four buckets the eval injects (Â§20.3) â€” so the Critic is graded on catching the precise failure modes it was prompted to look for, with the answer pre-seeded into `seasonality_calendar` (see next finding). Who critiques the Critic? Nobody. Majority-of-correlated-samplers is not adversarial robustness.

Fundamental. No oracle; self-grading by correlated models.

[HIGH] The seasonality/data-quality buckets leak their answers through pre-seeded memory

Claim attacked. Seasonality decoys and data-quality scenarios "must NOT flag it (Critic must refute)" (Â§20.3); the Critic "queries `seasonality_calendar` â€¦ if the observed change is within `expected_mag` of a calendar entry, the finding is DROP'ped" (Â§22.3).

Why it fails. Â§22.3 states the `seasonality_calendar` is "seeded onceâ€¦ the dataset spans 2020-11-01â†'2021-01-31, so Black Friday 2020 and the December peak/January trough are pre-loaded." Now look at the decoy scenarios: S033 post-holiday January trough, S034 Black Friday, S037 Christmas peak, S038 New Year â€" these are *the exact calendar entries pre-loaded into memory*. The Critic isn't reasoning that a swing is seasonal; it is doing a date-range lookup against a table that was hand-populated with the answer. The `launch_calendar` (Â§22.3, example "new /sale landing page") similarly pre-loads the mechanism for S006 (/sale landing page) and S023 (PDP change). The benchmark is testing whether a `JOIN` on a date overlaps â€" trivially passable â€" and dressing it as "the Critic refutes via seasonality_calendar." Take the seeded calendar away (i.e., production with no curated calendar) and the seasonality-decoy accuracy collapses, because there is nothing to look up.

Fundamental. Answers pre-seeded into the lookup table.

[HIGH] "Deterministic FSM with model-driven nodes" is a contradiction that hides where the answer is decided

Claim attacked. "Helios is **not** an autonomous free-roaming agent. It is a **deterministic finite state machine**" (AGENT_ARCHITECTURE.md Â§1); determinism is sold as Trust Pillar 2.

Why it fails. Â§11 of the same doc quietly concedes it: "LLM nodes are run at temperature 0 but are **not** byte-deterministic." The FSM determinism covers only *which stage runs next* â€" PLANâ†'MONITORâ†'DECOMPOSE â€" which is the trivial part. The *diagnosis itself* is whatever the stochastic Opus node decides: which slice to drill (Diagnose's best-first ordering depends on the model choosing what to expand within `MAX_BRANCHING=4` / `MAX_DEPTH=3`, Â§9), which causal story to tell, whether the Critic feels skeptical today. The load-bearing decisions are non-deterministic; the bookkeeping around them is deterministic. Selling "deterministic FSM" as a trust pillar conflates reproducible *control flow* with reproducible *answers*, and the latter is the only one that matters for a tool whose output is a causal claim. Two runs on identical frozen data can yield different root causes and different dollar figures â€" fatal for an "always-on autonomous" product whose runs all see the same static export.

Fundamental. Stochastic nodes decide the answer.

[HIGH] The <=45% baseline is a strawman engineered to be beaten

Claim attacked. "the naive baselineâ€¦ ranks by `|delta|`, and declares the single largest-magnitude segment the root causeâ€¦ scores **~45% top-1**" â€" "a near-doubling, which is the project's central empirical claim" (Â§20.5).

Why it fails. The baseline was chosen *because* the benchmark is dominated by buckets it must lose. 12 of 50 scenarios are pure mix-shift (S011â€"S020) where "largest absolute delta" is designed to be fooled, and the Simpson's-paradox adversarial cases (S014, S018) are constructed specifically so the naive method points at the wrong cell. The "~45%" isn't measured â€" it's asserted ("it gets pure single-segment rate cases right and most mix cases wrong," Â§20.5), i.e. back-computed from the bucket mix. A serious baseline (per-segment rate-delta ranking, or a chi-square contribution test, or literally "decompose then rank by rate term") would close most of the gap without any of Helios's 7-agent apparatus. Beating a baseline you wrote to lose, on data you injected with your own algebra, is not "the project's central empirical claim" â€" it is circular twice over.

Fixable. Pick an honest, decomposition-aware baseline.

[HIGH] Cost and latency of 7 agents (3 Opus) + tree + Critic re-query loop cannot credibly fit <5 min / and is unmeasured

Claim attacked. "Time-to-diagnosis **<5 min/run**" (Â§5.2) and the results table "Time: 4m12s/run" (Â§20.9); "This split keeps token cost inside budget" (Â§19.1).

Why it fails. The results table prints "4m12s" as if measured â€" it is fabricated; no code exists (CLAUDE.md Â§10: "No code yet"). Count the real critical path: Orchestrator (Opus) â†' Monitor

(Sonnet, one series per metric over ~22 canonical metrics) â†’ Decompose (Sonnet) â†’ Diagnose (Opus) walking a tree up to depth 3 Ã— branching 4 = up to ~64 nodes, *each* doing build_queryâ†’dry_runâ†’run_queryâ†’significance_testâ†’decompose_change (â‰¥5 sequential tool round-trips per node) â†’ Critic (Opus) per finding running its own re-queries â†’ up to `MAX_REFUTE_ROUNDS=2` *round trips back to Diagnose* (Â§19.3) â†’ Prescribe â†’ Narrator. Each Opus turn with tool use is multiple seconds; the Diagnose tree alone is hundreds of sequential model+tool round-trips because best-first search is inherently serial (you must see node N's decomposition before choosing N+1). The BYTE_BUDGET=5 GiB is enforced (Â§4.2) but there is no token budget enforced anywhere, and 3 Opus agents in a refute loop is the most expensive possible topology for a batch job that re-reads the same 3-month static table every run.

High (Fundamental for the latency claim). Serial tree + Opus refute loop.

[HIGH] Faithfulness is checked by an LLM judge â€” the same failure mode it is meant to catch

Claim attacked. "Faithfulnessâ€¦ does Narrator's prose claim match the SQL evidenceâ€¦ (entailment check)" target ">= 0.95"; "Critic-as-judge entailment pass that flags any sentence not entailed by the evidence bundle" (Â§20.4, Â§20.7).

Why it fails. Half the faithfulness check is a deterministic rule (every number maps to a tool-output hash, Â§20.7 check 1) â€” fine, but that only catches invented *numbers*, never invented *reasoning*. The other half, entailment, is "Critic-as-judge" â€” an LLM grading whether an LLM's prose is entailed by evidence. LLM-as-judge for faithfulness is exactly where models are known to be sycophantic and to rubber-stamp fluent, plausible prose. The dangerous output of Helios is a *fluent, plausible, wrong causal story over correct numbers* (see the iOS-SDK / shipping-calculator confabulations in S021/S024) â€” and that is precisely the output an LLM entailment judge is least able to flag, because the prose *sounds* entailed. The 0.95 target is again unmeasured and self-graded.

Fundamental. LLM judge can't catch fluent-wrong reasoning.

[MEDIUM] BigQuery + vector-store memory is massive over-engineering for a single-tenant batch job on 3 months of frozen data

Claim attacked. "Memory is split across BigQuery tablesâ€¦ and a **vector store** (embeddings of past findings)â€¦ `recall_prior` runs a hybrid query â€” exact filterâ€¦ plus a vector ANN searchâ€¦ decayed by `exp(-age_days/60)`" (Â§22.1); `HALF_LIFE=60d` "matching the ~3-month dataset window" (Â§9, Â§22.1).

Why it fails. The data is static, historical, single-store, ~3 months. Every "scheduled run" sees the identical frozen export, so "learns across runs" (Â§22) has nothing to learn from â€” re-running the

same window yields the same findings, and a 60-day half-life on a 92-day dataset means priors decay before a second meaningful cohort even exists. Diagnosis history will hold a handful of rows; ANN vector search over a few dozen findings is theatre â€” a `WHERE metric=? AND dimension_slice=?` exact match (which Â§22.1 *also* does) is sufficient and the embedding column is dead weight. The vector store adds an env var (`HELIOS_VECTOR_STORE_URI`), an upsert path, a recall path, and a similarity-decay formula to solve a problem that does not exist at this scale.

Fixable. Drop the vector store; a table suffices.

[MEDIUM] MCP (5 servers, two transports, JSON-RPC error taxonomy) is over-abstraction where in-process function calling would do

Claim attacked. "5 MCP servers" as architecture; the 14-code JSON-RPC error taxonomy (Â§5), dual transport (stdio + streamable-HTTP, Â§3), per-server `server_version` manifests, bearer-token auth on warehouse-mcp (Â§3).

Why it fails. Four of five servers (semantic, stats, experiment, report) run as local stdio subprocesses with "no warehouse credentials" and are "pure functions of their inputs" (Â§3, Â§8) â€” i.e., they are ordinary Python libraries wrapped in JSON-RPC framing for no isolation benefit, since they share the host and trust boundary anyway. The genuine grounding guarantee (the model never holds a raw-SQL handle) is achieved by the *agent tool allow-list* (Â§10), not by MCP-the-protocol; you get the identical guarantee by registering plain Python tool functions with the Agent SDK. The MCP layer buys serialization overhead, a bespoke error-code registry, subprocess lifecycle management, and a `mcp_servers.yaml` to maintain â€” for a single-process batch pipeline. The one server that *does* need a boundary (warehouse-mcp, the BigQuery client) could have been the sole tool; instead the doc spins up four credential-free subprocesses to look like a platform.

Fixable. Collapse stdio servers to in-process tools.

[MEDIUM] Allow-list and registry drift contradict the "single source of truth" discipline the trust pillars depend on

Claim attacked. "the Â§10 doc is the single source of truth" for tools (AGENT_ARCHITECTURE.md Â§2.1); "registry path = the reconciled canonical `./models/semantic/semantic_models.yml` " (MCP_ARCHITECTURE.md Â§3, Â§6.2 binding).

Why it fails. The grounding pillar's whole promise is that names resolve against one governed registry. Yet the docs point `semantic-mcp` 's `registry:` at `models/semantic/semantic_models.yml` (MCP_ARCHITECTURE.md Â§3 config, CLAUDE.md Â§5), while the artifact that actually exists and "passed a referential-integrity LINT" is

`semantic_layer.yaml` â€” two different filenames for the keystone. AGENT_ARCHITECTURE.md Â§13's exit gate and Â§20.9's results table say **34 scenarios**; the shipped `scenarios.yaml` header declares **50** across 7 buckets (S001â€“S050). The Bible Â§20.3 table sums to 34 with bucket counts (6/6/4/4/4/6/4) that do not match the file's (10/10/6/6/6/6/7). If the team cannot keep the *count of its own benchmark* or the *filename of its own keystone registry* consistent across three docs, the "0 hallucinated columns via the single canonical registry" guarantee is resting on coordination discipline the project has already demonstrably failed.

Fixable. Reconcile names/counts; but it indicts the discipline.
